

Biorthogonality, Step-Indexing and Compiler Correctness

Nick Benton

Microsoft Research
nick@microsoft.com

Chung-Kil Hur

University of Cambridge
ckh25@cam.ac.uk

Abstract

We define logical relations between the denotational semantics of a simply typed functional language with recursion and the operational behaviour of low-level programs in a variant SECD machine. The relations, which are defined using biorthogonality and step-indexing, capture what it means for a piece of low-level code to implement a mathematical, domain-theoretic function and are used to prove correctness of a simple compiler. The results have been formalized in the Coq proof assistant.

Categories and Subject Descriptors F.3.1 [Logics and Meanings of Programs]: Specifying and Verifying and Reasoning about Programs—Mechanical verification, Specification techniques; F.3.2 [Logics and Meanings of Programs]: Semantics of Programming Languages—Denotational semantics, Operational semantics; F.3.3 [Logics and Meanings of Programs]: Studies of Program Constructs—Type structure, Functional constructs; D.3.4 [Programming Languages]: Processors—Compilers; D.2.4 [Software Engineering]: Software / Program Verification—Correctness proofs, Formal methods

General Terms Languages, theory, verification

Keywords Compiler verification, denotational semantics, biorthogonality, step-indexing, proof assistants

1. Introduction

Proofs of compiler correctness have been studied for over forty years (McCarthy and Painter 1967; Dave 2003) and have recently been the subject of renewed attention, firstly because of increased interest in security and certification in a networked world and secondly because of advances in verification technology, both theoretical (e.g. separation logic, step-indexed logical relations) and practical (e.g. developments in model checking and improvements in interactive proof assistants).

There are many notions of correctness or safety that one might wish to establish of a compiler. For applying language-based techniques in operating systems design, as in proof-carrying code, one is primarily interested in broad properties such as type-safety, memory-safety or resource-boundedness. Although these terms are widely used, they are subject to a range of interpretations. For example, type-safety sometimes refers to a simple syntactic notion ('is the generated code typable using these rules?') and sometimes

to a deeper semantic one ('does the observable behaviour of the code satisfy this desirable property?'). In previous work (Benton 2006; Benton and Zarfaty 2007; Benton and Tabareau 2009), we have looked at establishing type-safety in the latter, more semantic, sense. Our key notion is that a high-level type translates to a low-level specification that should be satisfied by any code compiled from a source language phrase of that type. These specifications are inherently relational, in the usual style of PER semantics, capturing the meaning of a type A as a predicate on low-level heaps, values or code fragments *together* with a notion of A -equality thereon. These relations express what it means for a source-level abstraction (e.g. functions of type $A \rightarrow B$) to be respected by low-level code (e.g. 'taking A -equal arguments to B -equal results'). A crucial property of our low-level specifications is that they are defined in terms of the behaviour of low-level programs; making no reference to any intensional details of the code produced by a particular compiler or the grammar of the source language. Of course, the specifications *do* involve low-level details of data representations and calling conventions – these are part of the interface to compiled code – but up to that, code from any source that behaves sufficiently *like* code generated by the compiler should meet the specification, and this should be independently verifiable.

Ideally, one might wish to establish the sense in which a compilation scheme is *fully abstract*, meaning that the compiled versions of two source phrases of some type are in the low-level relation interpreting that type iff the original source phrases are contextually equivalent. If low-level specifications are used for checking linking between the results of compiling open programs and code from elsewhere¹ and full abstraction does not hold, then source level abstractions become 'leaky': reasoning about equivalence or encapsulation at the source level does not generally translate to the target, which can lead to unsound program transformations in optimizing compilers and to security vulnerabilities (Abadi 1998; Kennedy 2006). Ahmed and Blume (2008) also argue that fully abstract translation should be the goal, and prove full abstraction for (source to source) typed closure conversion for a polymorphic lambda calculus with recursive and existential types. Later on we will say something about why we believe that 'full' full abstraction may not, in practice, be quite the right goal, but we certainly *do* want 'sufficiently abstract' compilation, i.e. the preservation of the reasoning principles that we actually use in an optimizing compiler or in proving security properties.

The low-level relations of our previous work are not, however, really even sufficiently abstract, having roughly comparable power to a denotational semantics in continuation-passing style (CPS). This is a very strong and useful constraint on the behaviour of machine-code programs, but does not suffice to prove all the equa-

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ICFP'09, August 31–September 2, 2009, Edinburgh, Scotland, UK.
Copyright © 2009 ACM 978-1-60558-332-7/09/08...\$5.00

¹ This is obviously important for foreign function interfaces and multilanguage interoperability, but can be an issue even for separate compilation using the same compiler. It also covers the simpler and ubiquitous case of handcrafted implementations of standard library routines.

tions we might like between low-level programs: even something as simple as the commutativity of addition does not hold for arbitrary integer computations (just as it doesn't in a lambda calculus with control), even though it does in our pure source language. To understand how to refine our low-level relations further, it is natural to look at logical relations between low-level code and elements of the domains arising in a standard, direct-style, denotational model of our language, which is what we'll do here. Given such a typed relation between high-level semantics and low-level programs, a low-level notion of typed equivalence can be generated by considering pairs of low-level programs that are related to some common high-level denotational value.

The relations we define will establish a full functional correctness theorem for a simple compiler, not merely a semantic type safety theorem. Just as for type-safety, there are several approaches to formulating such correctness theorems in the literature. A common one, used for example by Leroy (2006), is to define an operational semantics for both high- and low-level languages and then establish a simulation (or bisimulation) result between source programs and their compiled versions, allowing one to conclude that if a closed high-level program terminates with a particular observable result, then its compiled version terminates with the same result, and often the converse too (Hardin et al. 1998; Leroy and Grall 2009). The limitation of these simulation-based theorems is that they are not as compositional (modular) or extensional (behavioural) as we would like, in that they only talk about the behaviour of compiled code in contexts that come from the same compiler, and usually specify a fairly close correspondence between the (non-observable) intermediate states of the source and target. We would rather have maximally permissive specifications that capture the full range of pieces of low-level code that, up to observations, behave like, or *realize*, a particular piece of high-level program. A slogan here is that 'the ends justify the means': we wish to allow low-level programs that extensionally get the right answers, whilst intensionally using any means necessary.

The main results here will involve logical relations between a cpo-based denotational semantics of a standard CBV lambda calculus with recursion and programs in an extended SECD-like target, chosen to be sufficiently low-level to be interesting, yet simple enough that the important ideas are not lost in detail. The relations will involve both biorthogonality and step-indexing, and in the next section we briefly discuss these constructions in general terms, before turning to the particular use we make of them.

The results in the paper have been formalized and proved in the Coq proof assistant, building on a formalization of domain theory and denotational semantics that we describe elsewhere (Benton et al. 2009). The scripts are available from the authors' webpages.

2. Orthogonality and Step-Indexing

2.1 Biorthogonality

Biorthogonality is a powerful and rather general idea that has been widely used in different kinds of semantics in recent years, beginning with the work of Pitts and Stark (1998) and of Krivine (1994). One way of understanding the basic idea is as a way of 'contextualizing' properties of parts of a system, making them compositional and behavioural. In the unary case, we start with some set of systems $\mathcal{S}$ (e.g. configurations of some machine, lambda terms, denotations of programs, processes) and some predicate $\mathcal{O} \subseteq \mathcal{S}$, which we call an observation, over these systems (e.g. those that execute without error, those that terminate, those that diverge). Then there is some way of combining, or plugging, program fragments $p \in \mathcal{P}$ in whose properties we are interested (bits of machine code, terms, denotations of terms, processes) with complementary contexts $c \in \mathcal{C}$ (frame stacks, terms with holes, continuations, other processes) to

yield complete systems. The plugging $- \circ - : \mathcal{P} \times \mathcal{C} \rightarrow \mathcal{S}$ might be effected by appending bits of program, substituting terms, applying continuations to arguments or composing processes in parallel. In any such situation, there is a contravariant map $(\cdot)^\perp : \mathbb{P}(\mathcal{P}) \rightarrow \mathbb{P}(\mathcal{C})$ given by

$$P^\perp = \{c \in \mathcal{C} \mid \forall p \in P, p \circ c \in \mathcal{O}\}$$

and a homonymous one in the other direction, $(\cdot)^\perp : \mathbb{P}(\mathcal{C}) \rightarrow \mathbb{P}(\mathcal{P})$

$$C^\perp = \{p \in \mathcal{P} \mid \forall c \in \mathcal{C}, p \circ c \in \mathcal{O}\}$$

yielding a contravariant Galois connection, so that, amongst many other things, $(\cdot)^{\perp\perp}$ is a closure operator (inflationary and idempotent) on $\mathbb{P}(\mathcal{P})$, and that any set of the form $C^\perp$ is $(\cdot)^{\perp\perp}$ -closed.² The binary version of this construction starts with a binary relation (e.g. an equivalence relation) on $\mathcal{S}$ and proceeds in the obvious way.

For compiler correctness, we want the interpretations of source-level types or, indeed, source-level values, to be compositional and extensional: properties of low-level program fragments that we can check independently and that make statements about the observable behavior of the complete configurations that arise when we plug the fragments into 'appropriate' contexts. This set of 'appropriate' contexts can be thought of as a set of tests: a low-level fragment is in the interpretation of a source type, or correctly represents a source value, just when it passes all these tests. Thus these low-level interpretations will naturally be $(\cdot)^{\perp\perp}$ -closed sets. For a simply typed source language, we can define these sets by induction on types, either positively, starting with an over-intensional set and then taking its $(\cdot)^{\perp\perp}$ -closure, or negatively, by first giving an inductive definition of a set of contexts and then taking its orthogonal. See Vouillon and Melliès (2004) for more on the use of biorthogonality in giving semantics to types.

2.2 Step-Indexing

Logical predicates and relations can be used with many different styles of semantics. When dealing with languages with recursion and working denotationally, however, we often need extra admissibility properties, such as closure under limits of ω -chains. Operational logical relations for languages with recursion generally need to satisfy some analogue of admissibility. One such that has often been used with operational semantics based on lambda terms (Plotkin 1977; Pitts and Stark 1998) considers replacing the recursion construct $\text{rec } f x = M$ (or fixpoint combinator) with a family of finite approximations: $\text{rec}_n f x = M$ for $n \in \mathbb{N}$, unfolds the recursive function n times in M and thereafter diverges. Appel and McAllester (2001) introduced step-indexed logical relations, which have since been refined and successfully applied by various authors to operational reasoning problems for both high and low level languages, many of which involve challenging language features (Ahmed 2006; Appel et al. 2007; Benton and Tabareau 2009; Ahmed et al. 2009). Step-indexing works with small-step operational semantics and $\mathbb{N}$ -indexed sets of values, with $(n, v) \in P$ (or $v \in P_n$) meaning 'value v has property P for n steps of reduction'. An interesting feature of step-indexing is that one usually works directly with this family of approximations; the limits that one feels are being approximated (like $\{v \mid \forall n, v \in P_n\}$) do not play much of a rôle.

2.3 On Using Both

Amongst the useful properties of the operational $(\cdot)^{\perp\perp}$ -closure operation used by Pitts and Stark (1998) is that it yields admissible

²Pitts and Stark, and some other non-Gallic authors, tend to write $(\cdot)^\top$ rather than $(\cdot)^\perp$.

relations (in the rec_n sense). The same is true of its natural denotational analogue (Abadi 2000). Our earlier work on low-level interpretations of high-level types (Benton and Tabareau 2009) used both step-indexing and orthogonality, but there was some question as to whether the step-indexing was really necessary. Maybe our closed sets are automatically already appropriately ‘admissible’, just by construction, and there is no need to add extra explicit indexing? Slightly to our surprise, it turns out that there *are* good reasons to use both $(\cdot)^{\perp\perp}$ -closure and step-indexing, for which we now try to give some intuition.

The aim is to carve out interpretations of high-level types and values as ‘well-behaved’ subsets of low-level, untyped programs. The essence of these interpretations generally only depends upon these well-behaved subsets: we’ll (roughly) interpret a function type $A \rightarrow B$ as the set of programs that when combined with a good argument of type A and a good continuation expecting something of type B , yield good behaviour. So exactly what range of impure, potentially type-abstraction violating, operations are available in the untyped language (the range of ‘any means necessary’ above) does not seem to affect the definitions or key results. Programs that use low-level features in improper ways will simply not be in the interpretations of high-level entities, and nothing more needs to be said. For a simply-typed total language *without* recursion, this intuition is quite correct: a Krivine-style realizability interpretation is essentially unaffected by adding extra operations to the untyped target. Even though orthogonality introduces quantification over bigger sets of contexts, nothing relies explicitly on properties of the untyped language as a whole.

In the presence of recursion, the situation changes. The fact that Pitts and Stark’s $(\cdot)^{\perp\perp}$ -closed relations are admissible depends on a ‘compactness of evaluation’ result, sometimes called an ‘unwinding theorem’, saying that any complete program p terminates iff there is some n such that for all $m \geq n$, p with all the rec_m s replaced by rec_m s terminates, which is clearly a global property of all untyped programs. In the denotational case, attention is already restricted to operations that can be modelled by continuous functions, i.e. ones that behave well with respect to approximation, in the chosen domains. But realistic targets often support operations that can violate these global properties. Examples of such egregiously non-functional operations include the ‘reflection’ features of some high-level languages (such as Java or C#) or, more interestingly, the ability of machine code programs to switch on code pointers or read executable machine instructions.³ We have found that the presence of such seriously non-functional operations does not just make the proofs harder, but can actually make the ‘natural’ theorems false. Appendix A shows how the addition of equality testing on lambda terms to an untyped lambda calculus breaks a ‘standard’ syntactic interpretation of simple types as sets of untyped terms in the presence of term-level recursion in the source.

Fortunately, as we will see, step-indexing sidesteps this problem. In place of appealing to a global property that holds of all untyped programs, we build a notion of approximation, and the requirement to behave well with respect to it, directly into the definition of our logical relations. In fact, we will also do something very similar on the denotational side, closing up explicitly under limits of chains.

³ Real implementations, even of functional languages, can make non-trivial use of such features. For example, interpreting machine instructions that would normally be executed in order to advance to a safe-point for interruption, building various runtime maps keyed on return addresses, doing emulation, JIT-compilation or SFI.

3. Source Language

Our high-level language PCF_v is a conventional simply-typed, call-by-value functional language with types built from integers and booleans by products and function spaces, with type contexts Γ defined in the usual way:

$$\begin{aligned} t &:= \text{Int} \mid \text{Bool} \mid t \rightarrow t' \mid t \times t' \\ \Gamma &:= x_1 : t_1, \dots, x_n : t_n \end{aligned}$$

We separate syntactic values (canonical forms), ranged over by V , from general expressions, ranged over by M and restrict the syntax to ANF, with explicit sequencing of evaluation by let and explicit inclusion of values into expressions by $[\cdot]$.

The typing rules for values and for expressions are shown in Figure 1. Note that there are really two forms of judgement, but we refrain from distinguishing them syntactically. The symbol $\star$ stands for an arbitrary integer-valued binary operation on integers, whilst $>$ is a representative boolean-valued one.

PCF_v has the obvious CBV operational semantics, which we elide here, and a conventional, and computationally adequate, denotational semantics in the category of ω -cpo and continuous maps, which we now briefly summarize to fix notation. Types and environments are interpreted as cpos:

$$\begin{aligned} \llbracket \text{Int} \rrbracket &\stackrel{\text{def}}{=} \mathbb{N} \\ \llbracket \text{Bool} \rrbracket &\stackrel{\text{def}}{=} \mathbb{B} \\ \llbracket t \rightarrow t' \rrbracket &\stackrel{\text{def}}{=} \llbracket t \rrbracket \Rightarrow \llbracket t' \rrbracket_{\perp} \\ \llbracket x_1 : t_1, \dots, x_n : t_n \rrbracket &\stackrel{\text{def}}{=} \llbracket t_1 \rrbracket \times \dots \times \llbracket t_n \rrbracket \end{aligned}$$

where $\Rightarrow$ is the cpo of continuous functions and $\times$ is the Cartesian product cpo. Typing judgements for values and expressions are then interpreted as continuous maps:

$$\begin{aligned} \llbracket \Gamma \vdash V : t \rrbracket &: \llbracket \Gamma \rrbracket \rightarrow \llbracket t \rrbracket \\ \llbracket \Gamma \vdash M : t \rrbracket &: \llbracket \Gamma \rrbracket \rightarrow \llbracket t \rrbracket_{\perp} \end{aligned}$$

defined by induction. So, for example

$$\begin{aligned} &\llbracket \Gamma \vdash \text{Fix } f x = M : A \rightarrow B \rrbracket \rho \\ &= \mu d_f. \lambda d_x \in \llbracket A \rrbracket. \llbracket \Gamma, f : A \rightarrow B, x : A \vdash M : B \rrbracket (\rho, d_f, d_x) \end{aligned}$$

We write $[\cdot] : D \rightarrow D_{\perp}$ for the unit of the lift monad. We elide the full details of the denotational semantics as they are essentially the same as those found in any textbook, such as that of Winskel (1993). The details can also be found, along with discussion of the Coq formalization of the semantics, in Benton et al. (2009).

4. Target Language and Compilation

4.1 An SECD Machine

The low-level target is a variant of an SECD virtual machine (Landin 1964). We have chosen such a machine rather than a lower-level assembly language, such as that of our previous work, so as to keep the formal development less cluttered with detail. But we are emphatically not interested in the SECD machine as something that is inherently “for” executing programs in a language like our source. We have included an equality test instruction, Eq , that works on arbitrary values, including closures, so a counterexample to a naive semantics of types like that in Appendix A can be constructed. Furthermore, the logical relations we present have been carefully constructed with applicability to lower-level machines in mind.

Values:	$[TVAR] \frac{}{\Gamma, x : t \vdash x : t} \quad [TBOOL] \frac{}{\Gamma \vdash b : \text{Bool}} (b \in \mathbb{B}) \quad [TINT] \frac{}{\Gamma \vdash n : \text{Int}} (n \in \mathbb{N})$
Expressions:	$[TFIX] \frac{\Gamma, f : t \rightarrow t', x : t \vdash M : t'}{\Gamma \vdash \text{Fix } f x = M : t \rightarrow t'} \quad [TP] \frac{\Gamma \vdash V_i : t_i (i = 1, 2)}{\Gamma \vdash \langle V_1, V_2 \rangle : t_1 \times t_2}$ $[TVAL] \frac{\Gamma \vdash V : t}{\Gamma \vdash [V] : t} \quad [TLET] \frac{\Gamma \vdash M : t \quad \Gamma, x : t \vdash N : t'}{\Gamma \vdash \text{let } x = M \text{ in } N : t'}$ $[TAPP] \frac{\Gamma \vdash V_1 : t \rightarrow t' \quad \Gamma \vdash V_2 : t}{\Gamma \vdash V_1 V_2 : t'} \quad [TIF] \frac{\Gamma \vdash V : \text{Bool} \quad \Gamma \vdash M_1 : t \quad \Gamma \vdash M_2 : t}{\Gamma \vdash \text{if } V \text{ then } M_1 \text{ else } M_2 : t}$ $[TOP] \frac{\Gamma \vdash V_1 : \text{Int} \quad \Gamma \vdash V_2 : \text{Int}}{\Gamma \vdash V_1 \star V_2 : \text{Int}} \quad [TGT] \frac{\Gamma \vdash V_1 : \text{Int} \quad \Gamma \vdash V_2 : \text{Int}}{\Gamma \vdash V_1 > V_2 : \text{Bool}}$ $[TFST, TSND] \frac{\Gamma \vdash V : t_1 \times t_2}{\Gamma \vdash \pi_i(V) : t_i (i = 1, 2)}$

Figure 1. Typing rules for PCF_v

The inductive type *Instruction*, ranged over by i , is defined by

$i := \text{Swap} \mid \text{Dup} \mid \text{PushV } n \mid \text{Op } \star \mid \text{PushC } c \mid \text{PushRC } c \mid \text{App} \mid \text{Ret} \mid \text{Sel } (c_1, c_2) \mid \text{Join} \mid \text{MkPair} \mid \text{Fst} \mid \text{Snd} \mid \text{Eq}$

where c ranges over *Code*, the set of lists of instructions, n ranges over integers, and $\star$ over binary operations on integers. The set *Value* of runtime values, ranged over by v is defined by

$v := \underline{n} \mid \text{CL}(e, c) \mid \text{RCL}(e, c) \mid \text{PR}(v_1, v_2)$

where e ranges over *Env*, defined to be *list Value*. So a *Value* is either an integer literal, a closure containing an environment and some code, a recursive closure, or a pair values. We also define

$\text{Stack} = \text{list Value}$
 $\text{Dump} = \text{list } (\text{Code} \times \text{Env} \times \text{Stack})$
 $\text{CESD} = \text{Code} \times \text{Env} \times \text{Stack} \times \text{Dump}$

CESD is the set of configurations of our virtual machine. A configuration $\langle c, e, s, d \rangle$ comprises the code to be executed, c , the current environment, e , an evaluation stack s and a call stack, or dump, d .

The deterministic one-step transition relation $\mapsto$ between configurations is defined in Figure 2. There are many configurations with no successor, such as those in which the next instruction is *Swap* but the the stack depth is less than two; we say such configurations are *stuck* or *terminated*. (So there is no *a priori* distinction between normal and abnormal termination.) We write $\text{cesd} \mapsto^k$ to mean that the configuration cesd takes at least k steps without getting stuck, and say it diverges, written $\text{cesd} \mapsto^\omega$ if it can always take a step:

$$\text{cesd} \mapsto^\omega \stackrel{\text{def}}{\iff} (\forall k, \text{cesd} \mapsto^k)$$

Conversely, we say cesd terminates, and write $\text{cesd} \mapsto^* \not\downarrow$, if it does not diverge:

$$\text{cesd} \mapsto^* \not\downarrow \stackrel{\text{def}}{\iff} \neg(\text{cesd} \mapsto^\omega).$$

4.2 Compiling PCF_v to SECD

The compiler comprises two mutually-inductive functions mapping (typed) PCF_v values and expressions into *Code*. We overload $\langle \cdot \rangle$ for both of these functions, whose definitions are shown in Figure 3.

5. Logical Relations

In this section we define logical relations between components of the low-level SECD machine and elements of semantic domains, with the intention of capturing just when a piece of low-level code realizes a semantic object. In fact, there will be two relations, one defining when a low-level component approximates a domain element, and one saying when a domain element approximates a low-level component. These roughly correspond to the soundness and adequacy theorems that one normally proves to show a correspondence between an operational and denotational semantics, but are rather more complex.

Following the general pattern of biorthogonality sketched above, we work with (predicates on) substructures of complete configurations. On the SECD machine side, complete configurations are elements of *CESD*, whilst the substructures will be elements of *Value* and of *Comp*, which is defined to be $\text{Code} \times \text{Stack}$. If $v : \text{Value}$ then we define $\hat{v} : \text{Comp}$ to be $([], [v])$, the computation comprising an empty instruction sequence and a singleton stack with v on. Similarly, if $c : \text{Code}$ then $\hat{c} : \text{Comp}$ is $(c, [])$.

The basic plugging operation on the low-level side is $\cdot \bowtie \cdot$, taking an element of *Comp*, the computation under test, and an element of *CESD*, thought of as a context, and combining them to yield a configuration in *CESD*:

$$(c, s) \bowtie \langle c', e', s', d' \rangle = \langle c ++ c', e', s ++ s', d' \rangle$$

We also have an operation $\cdot \smile \cdot$ that appends an element of *Env* onto the environment component of a configuration:

$$e \smile \langle c', e', s', d' \rangle = \langle c', e ++ e', s', d' \rangle$$

5.1 Approximating Denotational By Operational

The logical relation expressing what it means for low-level computations to approximate denotational values works with step-indexed entities. We write $i\text{Value}$ for $\mathbb{N} \times \text{Value}$, $i\text{Comp}$ for $\mathbb{N} \times \text{Comp}$ and

$\langle \text{Swap} :: c, e, v_1 :: v_2 :: s, d \rangle$	$\mapsto$	$\langle c, e, v_2 :: v_1 :: s, d \rangle$
$\langle \text{Dup} :: c, e, v :: s, d \rangle$	$\mapsto$	$\langle c, e, v :: v :: s, d \rangle$
$\langle \text{PushV } n :: c, [v_1, \dots, v_k], s, d \rangle$	$\mapsto$	$\langle c, [v_1, \dots, v_k], v_n :: s, d \rangle$
$\langle \text{PushN } n :: c, e, s, d \rangle$	$\mapsto$	$\langle c, e, \underline{n} :: s, d \rangle$
$\langle \text{PushC } bod :: c, e, s, d \rangle$	$\mapsto$	$\langle c, e, \text{CL}(e, bod) :: s, d \rangle$
$\langle \text{PushRC } bod :: c, e, s, d \rangle$	$\mapsto$	$\langle c, e, \text{RCL}(e, bod) :: s, d \rangle$
$\langle \text{App} :: c, e, v :: \text{CL}(e', bod) :: s, d \rangle$	$\mapsto$	$\langle bod, v :: e', [], (c, e, s) :: d \rangle$
$\langle \text{App} :: c, e, v :: \text{RCL}(e', bod) :: s, d \rangle$	$\mapsto$	$\langle bod, v :: \text{RCL}(e', bod) :: e', [], (c, e, s) :: d \rangle$
$\langle \text{Op } \star :: c, e, \underline{n_2} :: \underline{n_1} :: s, d \rangle$	$\mapsto$	$\langle c, e, \underline{n_1} \star \underline{n_2} :: s, d \rangle$
$\langle \text{Ret} :: c, e, v :: s, (c', e', s') :: d \rangle$	$\mapsto$	$\langle c', e', v :: s', d \rangle$
$\langle \text{Sel } (c_1, c_2) :: c, e, v :: s, d \rangle$	$\mapsto$	$\langle c_1, e, s, (c, [], []) :: d \rangle$ (if $v \neq \underline{0}$)
$\langle \text{Sel } (c_1, c_2) :: c, e, \underline{0} :: s, d \rangle$	$\mapsto$	$\langle c_2, e, s, (c, [], []) :: d \rangle$
$\langle \text{Join} :: c, e, s, (c', e', s') :: d \rangle$	$\mapsto$	$\langle c', e, s, d \rangle$
$\langle \text{MkPair} :: c, e, v_1 :: v_2 :: s, d \rangle$	$\mapsto$	$\langle c, e, \text{PR}(v_2, v_1) :: s, d \rangle$
$\langle \text{Fst} :: c, e, \text{PR}(v_1, v_2) :: s, d \rangle$	$\mapsto$	$\langle c, e, v_1 :: s, d \rangle$
$\langle \text{Snd} :: c, e, \text{PR}(v_1, v_2) :: s, d \rangle$	$\mapsto$	$\langle c, e, v_2 :: s, d \rangle$
$\langle \text{Eq} :: c, e, v_1 :: v_2 :: s, d \rangle$	$\mapsto$	$\langle c, e, \underline{1} :: s, d \rangle$ (if $v_1 = v_2$)
$\langle \text{Eq} :: c, e, v_1 :: v_2 :: s, d \rangle$	$\mapsto$	$\langle c, e, \underline{0} :: s, d \rangle$ (if $v_1 \neq v_2$)

Figure 2. Operational Semantics of SECD Machine

Values:

$\langle x_1 : t_1, \dots, x_n : t_n \vdash x_i : t_i \rangle$	$=$	$[\text{PushV } i]$
$\langle \Gamma \vdash \text{true} : \text{Bool} \rangle$	$=$	$[\text{PushN } 1]$
$\langle \Gamma \vdash \text{false} : \text{Bool} \rangle$	$=$	$[\text{PushN } 0]$
$\langle \Gamma \vdash n : \text{Int} \rangle$	$=$	$[\text{PushN } n]$
$\langle \Gamma \vdash \langle V_1, V_2 \rangle : t_1 \times t_2 \rangle$	$=$	$\langle \Gamma \vdash V_1 : t_1 \rangle ++ \langle \Gamma \vdash V_2 : t_2 \rangle ++ [\text{MkPair}]$
$\langle \Gamma \vdash \text{Fix } f x = M : t \rightarrow t' \rangle$	$=$	$[\text{PushRC } (\langle \Gamma, f : t \rightarrow t', x : t \vdash M : t' \rangle ++ [\text{Ret}])] ++ [\text{App}]$

Expressions:

$\langle \Gamma \vdash [V] : t \rangle$	$=$	$\langle \Gamma \vdash V : t \rangle$
$\langle \Gamma \vdash \text{let } x = M \text{ in } N : t' \rangle$	$=$	$[\text{PushC } (\langle \Gamma, x : t \vdash N : t' \rangle ++ [\text{Ret}])] ++ \langle \Gamma \vdash M : t \rangle ++ [\text{App}]$
$\langle \Gamma \vdash V_1 V_2 : t' \rangle$	$=$	$\langle \Gamma \vdash V_1 : t \rightarrow t' \rangle ++ \langle \Gamma \vdash V_2 : t \rangle ++ [\text{App}]$
$\langle \Gamma \vdash \text{if } V \text{ then } M_1 \text{ else } M_2 : t \rangle$	$=$	$\langle \Gamma \vdash V : \text{Bool} \rangle ++ [\text{Sel } ((\langle \Gamma \vdash M_1 : t \rangle ++ [\text{Join}]), (\langle \Gamma \vdash M_2 : t \rangle ++ [\text{Join}]))]$
$\langle \Gamma \vdash V_1 \star V_2 : \text{Int} \rangle$	$=$	$\langle \Gamma \vdash V_1 : \text{Int} \rangle ++ \langle \Gamma \vdash V_2 : \text{Int} \rangle ++ [\text{Op } \star]$
$\langle \Gamma \vdash V_1 > V_2 : \text{Bool} \rangle$	$=$	$\langle \Gamma \vdash V_1 : \text{Int} \rangle ++ \langle \Gamma \vdash V_2 : \text{Int} \rangle ++ [\text{Op } (\lambda(n_1, n_2). n_1 > n_2 \supset 1 \mid 0)]$

Figure 3. Compiler for PCF_v

$i\text{CESD}$ for $\mathbb{N} \times \text{CESD}$ and define an Env -parameterized observation $\mathcal{O}e$ over pairs of indexed computations from $i\text{Comp}$ and indexed contexts from $i\text{CESD}$:

$$\mathcal{O}e(i, \text{comp})(j, \text{cesd}) \stackrel{\text{def}}{\iff} (\text{comp} \bowtie (e \smile \text{cesd})) \mapsto^{\min(i, j)}$$

So, given an environment e , $\mathcal{O}e$ holds of an indexed computation and an indexed context just when the configuration that results from appending the environment e and the code and stack from the computation onto the corresponding components of the context steps for at least the minimum of the indices of the $i\text{Comp}$ and the $i\text{CESD}$. We also define an observation on pairs of indexed values and indexed contexts by lifting values to computations:

$$\mathcal{O}e(i, v)(j, \text{cesd}) \stackrel{\text{def}}{\iff} \mathcal{O}e(i, \hat{v})(j, \text{cesd})$$

Now we follow the general pattern of orthogonality, but with some small twists. We actually have a collection of observations, indexed by environments, made over step-indexed components of configu-

rations. And each observation gives rise to two, closely related, Galois connections: one between predicates on (indexed) values and predicates on (indexed) contexts, and the other between those on (indexed) computations and (indexed) contexts. So there are four contravariant maps associated with each $e : \text{Env}$. Our definitions use these two of them:

$$\begin{aligned} \downarrow^e(\cdot) &: \mathbb{P}(i\text{Value}) \rightarrow \mathbb{P}(i\text{CESD}) \\ \downarrow^e(P) &= \{j\text{cesd} \mid \forall iv \in P, \mathcal{O}e\ iv\ j\text{cesd}\} \\ \uparrow^e(\cdot) &: \mathbb{P}(i\text{CESD}) \rightarrow \mathbb{P}(i\text{Comp}) \\ \uparrow^e(Q) &= \{i\text{comp} \mid \forall j\text{cesd} \in Q, \mathcal{O}e\ i\text{comp}\ j\text{cesd}\} \end{aligned}$$

To explain the notation: down arrows translate positive predicates (over values and computations) into negative ones (over contexts), whilst up arrows go the other way. We use single arrows for the operations relating to values and double arrows for those relating to computations.

Now we can start relating the low-level machine to the high-level semantics. If D is a cpo and $RD_i \subseteq \text{Value} \times D$ is a $\mathbb{N}$ -indexed relation between machine values and elements of D then define the indexed relation $[RD]_n \subseteq \text{Value} \times D_\perp$ by

$$[RD]_n = \{(v, dv) \mid \exists d \in D, [d] = dv \wedge (v, d) \in RD_n\}$$

If, furthermore, S is a cpo and $RS_i \subseteq \text{Env} \times S$ an indexed relation between machine environments and elements of S , then we define an indexed relation

$$\begin{aligned} (RS \overset{\Delta}{\rhd} RD_\perp)_i &\subseteq \text{Comp} \times (S \Rightarrow D_\perp) \\ &= \{(comp, df) \mid \forall k \leq i, \forall (e, de) \in RS_k, \\ &\quad (k, comp) \in \uparrow^e (\downarrow^e (\{(j, v) \mid (v, df\ de) \in [RD]_j\}))\} \end{aligned}$$

which one should see as the relational action of the lift monad, relative to a relation on environments. The definition looks rather complex, but the broad shape of the definition is ‘logical’: relating machine computations to denotational continuous maps just when RS -related environments yield $[RD]$ -related results. Then there is a little extra complication caused by threading the step indices around, but this is also of a standard form: computations are in the relation at i when they take k -related arguments to k -related results for all $k \leq i$. Finally, we use biorthogonality to close up on the right hand side of the arrow; rather than making an intensional direct-style definition that the computation ‘yields’ a value v that is related to the denotational application $df\ de$, we take the set of all such related results, flip it across to an orthogonal set of contexts with $\downarrow^e(\cdot)$ and then take that back to a set of computations with $\uparrow^e(\cdot)$.

A special case of indexed relations between machine environments and denotational values is that for the empty environment. We define $I_i \subseteq \text{Env} \times 1$, where 1 is the one-point cpo, by $I_i = \{(\llbracket \cdot \rrbracket, *)\}$. We can now define the ‘real’ indexed logical relation of approximation between machine values and domain elements

$$\leq_i^t \subseteq \text{Value} \times \llbracket t \rrbracket$$

where t is a type and i is a natural number index like this:

$$\begin{aligned} \leq_i^{\text{Int}} &= \{(\underline{n}, n) \mid n \in \mathbb{N}\} \\ \leq_i^{\text{Bool}} &= \{(0, \text{false})\} \cup \{(\underline{n+1}, \text{true}) \mid n \in \mathbb{N}\} \\ \leq_i^{t \times t'} &= \{(\text{PR}(v_1, v_2), (dv_1, dv_2)) \mid (v_1, dv_1) \in \leq_i^t \wedge \\ &\quad (v_2, dv_2) \in \leq_i^{t'}\} \\ \leq_i^{t \rightarrow t'} &= \{(f, df) \mid \forall k \leq i, \forall (v, dv) \in \leq_k^t, \\ &\quad (([\text{App}], [v, f]], \lambda * : 1. df\ dv) \in (I \overset{\Delta}{\rhd} (\leq^{t'}_\perp)_k)\} \end{aligned}$$

This says that machine integers approximate the corresponding denotational ones, the machine zero approximates the denotational ‘false’ value, and all non-zero machine integers approximate the denotational ‘true’ value, reflecting the way in which the low-level conditional branch instruction works and the way in which we compile source-level booleans. Pair values on the machine approximate denotational pairs pointwise.

As usual, the interesting case is that for functions. The definition says that a machine value f and a semantic function df are related at type $t \rightarrow t'$ if whenever v is related to dv at type t , the computation whose code part is a single application instruction and whose stack part is the list $[v, f]$ is related to the constantly $(df\ dv)$ function, of type $1 \Rightarrow \llbracket t' \rrbracket_\perp$, by the monadic lifting of the approximation relation at type t' .

Having defined the relation for values, we lift it to environments in the usual pointwise fashion. If Γ is $x_1 : t_1, \dots, x_n : t_n$ then $\leq_i^\Gamma \subseteq \text{Env} \times \llbracket \Gamma \rrbracket$ is given by

$$\leq_i^\Gamma = \{([v_1, \dots, v_n], (d_1, \dots, d_n)) \mid \forall l, (v_l, d_l) \in \leq_i^{t_l}\}$$

For computations in context, we define $\leq_i^{\Gamma, t} \subseteq \text{Comp} \times (\llbracket \Gamma \rrbracket \Rightarrow \llbracket t \rrbracket_\perp)$ using the monadic lifting again

$$\leq_i^{\Gamma, t} = (\leq_i^\Gamma \overset{\Delta}{\rhd} (\leq^t_\perp)_i)$$

These relations are antimonotonic in the step indices and monotone in the domain-theoretic order (we also switch to infix notation for relations at this point):

Lemma 1.

1. If $v \leq_i^t d$, $d \sqsubseteq d'$ and $j \leq i$ then $v \leq_j^t d'$.
2. If $e \leq_i^\Gamma \rho$, $\rho \sqsubseteq \rho'$ and $j \leq i$ then $e \leq_j^\Gamma \rho'$.
3. If $comp \leq_i^{\Gamma, t} f$, $f \sqsubseteq f'$ and $j \leq i$ then $comp \leq_j^{\Gamma, t} f'$.

The non-indexed versions of the approximation relations are then given by universally quantifying over the indices.

$$\begin{aligned} v \preceq^t d &\stackrel{\text{def}}{\iff} \forall i, v \leq_i^t d \\ e \preceq^\Gamma \rho &\stackrel{\text{def}}{\iff} \forall i, e \leq_i^\Gamma \rho \\ comp \preceq^{\Gamma, t} df &\stackrel{\text{def}}{\iff} \forall i, comp \leq_i^{\Gamma, t} df \end{aligned}$$

Note that the relation on computations extends that on values:

Lemma 2. If $v \preceq^t d$ then $\widehat{v} \preceq^{\llbracket t \rrbracket} (\lambda * : 1. [d])$.

5.2 Approximating Operational By Denotational

Our second logical relation captures what it means for a denotational value to be ‘less than or equal to’ a machine computation. This way around we will again use biorthogonality, but this time with respect to the observation of termination. This is intuitively reasonable, as showing that the operational behaviour of a program is at least as defined as some domain element will generally involve showing that reductions terminate. We will not use operational step-indexing to define the relation this way around, but an explicit admissible closure operation will play a similar rôle.

For $e \in \text{Env}$, $comp \in \text{Comp}$ and $cesd \in \text{CESD}$ our termination observation is defined by

$$\mathcal{T}\ e\ comp\ cesd \stackrel{\text{def}}{\iff} (comp \bowtie (e \smile cesd)) \mapsto^* \downarrow$$

which we again lift to values $v \in \text{Value}$:

$$\mathcal{T}\ e\ v\ cesd \stackrel{\text{def}}{\iff} \mathcal{T}\ e\ \widehat{v}\ cesd$$

and again the observations generate two e -parameterized Galois connections, one between predicates on values and predicates on contexts, and the other between predicates on computations and predicates on contexts. Once more we use two of the four maps:

$$\begin{aligned} \downarrow^e(\cdot) &: \mathbb{P}(\text{Value}) \rightarrow \mathbb{P}(\text{CESD}) \\ \downarrow^e(P) &= \{cesd \mid \forall v \in P, \mathcal{T}\ e\ v\ cesd\} \\ \uparrow^e(\cdot) &: \mathbb{P}(\text{CESD}) \rightarrow \mathbb{P}(\text{Comp}) \\ \uparrow^e(Q) &= \{comp \mid \forall cesd \in Q, \mathcal{T}\ e\ comp\ cesd\} \end{aligned}$$

to define a relational action for the lift monad. If S and D are cpos, $RS \subseteq \text{Env} \times S$ and $RD \subseteq \text{Value} \times D$, then define

$$\begin{aligned} (RS \overset{\Delta}{\rhd} RD_\perp) &\subseteq \text{Comp} \times (S \Rightarrow D_\perp) \\ &= \{(comp, df) \mid \forall (e, de) \in RS, \forall d, [d] = (df\ de) \\ &\quad \implies comp \in \uparrow^e(\downarrow^e(\{v \mid (v, d) \in RD\}))\} \end{aligned}$$

which follows a similar pattern to our earlier definition, in using biorthogonality on the right hand side of the arrow: starting with all values related to d , flipping that over to the set of contexts that terminate when plugged into any of those values and then coming back to the set of all computations that terminate when plugged into any of those contexts.

Now we can define the second logical relation

$$\succeq^t \subseteq \text{Value} \times \llbracket t \rrbracket$$

by induction on the type t :

$$\begin{aligned} \succeq^{\text{Int}} &= \{(\underline{n}, n) \mid n \in \mathbb{N}\} \\ \succeq^{\text{Bool}} &= \{(\underline{0}, \text{false})\} \cup \{(n+1, \text{true}) \mid n \in \mathbb{N}\} \\ \succeq^{t \times t'} &= \{(\text{PR}(v_1, v_2), (dv_1, dv_2)) \mid (v_1, dv_1) \in \succeq^t \wedge \\ &\quad (v_2, dv_2) \in \succeq^{t'}\} \\ \succeq^{t \rightarrow t'} &= \{(f, df) \mid \forall (v, dv) \in \succeq^t, \\ &\quad (([\text{App}], [v, f]), \lambda * : 1. df \, dv) \in (I \succeq^t (\succeq^{t'})_{\perp})\} \end{aligned}$$

This relation also lifts pointwise to environments. For $\Gamma = x_1 : t_1, \dots, x_n : t_n$ define $\succeq^{\Gamma} \subseteq \text{Env} \times \llbracket \Gamma \rrbracket$ by

$$\succeq^{\Gamma} = \{([v_1, \dots, v_n], (dv_1, \dots, dv_n)) \mid \forall l, (v_l, dv_l) \in \succeq^{t_l}\}$$

and then for computations in context, $\succeq^{\Gamma, t} \subseteq (\llbracket \Gamma \rrbracket \Rightarrow \llbracket t \rrbracket)_{\perp}$ is given by

$$\succeq^{\Gamma, t} = (\succeq^{\Gamma} \rightrightarrows (\succeq^t)_{\perp})$$

The $\succeq$ relations are all down-closed on the denotational side:

Lemma 3.

1. If $v \succeq^t d$ and $d' \sqsubseteq d$ then $v \succeq^t d'$
2. If $e \succeq^{\Gamma} \rho$ and $\rho' \sqsubseteq \rho$ then $e \succeq^{\Gamma} \rho'$
3. If $\text{comp} \succeq^{\Gamma, t} f$ and $f' \sqsubseteq f$ then $\text{comp} \succeq^{\Gamma, t} f'$

However, for a fixed v , it turns out that $\{d \mid v \succeq^t d\}$ is not always closed under taking limits of chains (see Appendix B for a more detailed explanation), which would prevent our compiler correctness proof going through in the case of recursion. We solve this problem by taking the Scott-closure. The closed subsets of a cpo D are those that are both down-closed and closed under limits of ω -chains. The closure $\text{Clos}(P)$ of a subset $P \subseteq D$ is the smallest closed subset of D containing P . So we define

$$\begin{aligned} v \succeq^t dv &\stackrel{\text{def}}{\iff} dv \in \text{Clos}(\{d \mid v \succeq^t d\}) \\ e \succeq^{\Gamma} de &\stackrel{\text{def}}{\iff} de \in \text{Clos}(\{d \mid e \succeq^{\Gamma} d\}) \\ \text{comp} \succeq^{\Gamma, t} df &\stackrel{\text{def}}{\iff} df \in \text{Clos}(\{d \mid \text{comp} \succeq^{\Gamma, t} d\}) \end{aligned}$$

Again, the relation for computations extends that for values:

Lemma 4. If $v \succeq^t d$ then $\widehat{v} \succeq^{\llbracket t \rrbracket, t} (\lambda * : 1. [d])$.

5.3 Realizability and Equivalence

Having defined the two logical relations, we can clearly put them together to define relations expressing that a machine value, environment or computation realizes a domain element:

$$\begin{aligned} v \models^t d &\stackrel{\text{def}}{\iff} v \succeq^t d \wedge v \succeq^t d \\ e \models^{\Gamma} de &\stackrel{\text{def}}{\iff} e \succeq^{\Gamma} de \wedge e \succeq^{\Gamma} de \\ \text{comp} \models^{\Gamma, t} df &\stackrel{\text{def}}{\iff} \text{comp} \succeq^{\Gamma, t} df \wedge \text{comp} \succeq^{\Gamma, t} df. \end{aligned}$$

and these relations naturally induce typed relations between machine components. We just give the case for computations

$$\Gamma \vdash \text{comp}_1 \sim \text{comp}_2 : t$$

$$\stackrel{\text{def}}{\iff} \exists df \in (\llbracket \Gamma \rrbracket \Rightarrow \llbracket t \rrbracket)_{\perp}, \text{comp}_1 \models^{\Gamma, t} df \wedge \text{comp}_2 \models^{\Gamma, t} df$$

and we overload this notation to apply to simple pieces of code too:

$$\Gamma \vdash c_1 \sim c_2 : t \stackrel{\text{def}}{\iff} \Gamma \vdash \widehat{c}_1 \sim \widehat{c}_2 : t$$

There seems no general reason to expect that the $\sim$ relations are transitive (though we have no concrete counterexample), so we

apply a transitive closure operation to get a low-level notion of equivalence $\sim$:

$$\Gamma \vdash \text{comp}_1 \sim \text{comp}_2 : t \stackrel{\text{def}}{\iff} \Gamma \vdash \text{comp}_1 \sim^+ \text{comp}_2 : t$$

So now we have a notion of what it means for a piece of SECD machine code to be in the semantic interpretation of a source language type, and what it means for two pieces of code to be equal when considered at that type. Clearly, we are going to use this to say something about the correctness of our compilation scheme, but note that the details of just what code the compiler produces have not really shown up at all in the definitions of our logical relations: it is only the interfaces to compiled code – the way in which integers, booleans and pairs are encoded and the way in which function values are tested via application – that are mentioned in the logical relation. In fact, equivalence classes of the $\sim$ relations can be seen as defining a perfectly good compositional ‘denotational’ semantics for the source language in their own right.

A feature of this semantics is that there are no statements about what (non-observable) intermediate configurations should look like. For example, we never say that when a function is entered with a call stack that looks like ‘ x ’, then eventually one reaches a configuration that looks like ‘ y ’. At the end of the day, all we ever talk about is termination and divergence of complete configurations, which we need to connect with the intended behaviour of closed programs of ground type; this we do by considering a range of possible external test contexts, playing the role of top-level continuations.

If $c \in \text{Code}$ then we say c diverges unconditionally if

$$\forall \text{cesd}, (\widehat{c} \bowtie \text{cesd}) \mapsto^{\omega}$$

The following says that if a piece of code realizes the denotational bottom value at any type, then it diverges unconditionally:

Lemma 5 (Adequacy for bottom). For any $c \in \text{Code}$ and type t , if

$$c \models^{\llbracket t \rrbracket, t} (\lambda * : 1. \perp_{\llbracket t \rrbracket})$$

then c diverges unconditionally.

For ground type observations, we say a computation comp converges to a particular integer value n if plugging it into an arbitrary context equiterminates with plugging n into that context:

$$\begin{aligned} \forall \text{cesd}, \widehat{n} \bowtie \text{cesd} \mapsto^* \downarrow &\implies \text{comp} \bowtie \text{cesd} \mapsto^* \downarrow \\ \wedge \widehat{n} \bowtie \text{cesd} \mapsto^{\omega} &\implies \text{comp} \bowtie \text{cesd} \mapsto^{\omega}. \end{aligned}$$

And we can then show that if a piece of code realizes a non-bottom element $[n]$ of $\llbracket \text{Int} \rrbracket_{\perp}$ in the empty environment, then it converges to n :

Lemma 6 (Ground termination adequacy). For any $c \in \text{Code}$, if

$$c \models^{\llbracket t \rrbracket, t} (\lambda * : 1. [n])$$

then $\widehat{c}$ converges to n .

Adequacy also holds for observation at the boolean type, with a definition of convergence to a value b that quantifies over those test contexts cesd that terminate or diverge uniformly for all machine values representing b .

We finally show the compositionality of our realizability semantics.

Lemma 7 (Compositionality for application). For any $cf, cx \in \text{Code}$ and $df \in (\llbracket \Gamma \rrbracket \Rightarrow (\llbracket t' \rrbracket)_{\perp})_{\perp}, dx \in \llbracket \Gamma \rrbracket \Rightarrow \llbracket t \rrbracket_{\perp}$, if

$$cf \models^{\Gamma, t \rightarrow t'} df \wedge cx \models^{\Gamma, t} dx$$

then

$$cf \text{ ++ } cx \text{ ++ } [\text{App}] \models^{\Gamma, t'} \lambda de : \llbracket \Gamma \rrbracket. (df \, de) \vee (dx \, de)$$

where $\vee$ denotes the lifted (Kleisli) application.

6. Applications

In this section, we illustrate the kind of results one can establish using the logical relations of the previous section.

6.1 Compiler Correctness

Our motivating application was establishing the functional correctness of the compiler that we presented earlier.

Theorem 1.

1. For all Γ, V, t , if $\Gamma \vdash V : t$ then

$$\langle \Gamma \vdash V : t \rangle \preceq^{\Gamma, t} [\langle \Gamma \vdash V : t \rangle]$$
2. For all Γ, M, t , if $\Gamma \vdash M : t$ then

$$\langle \Gamma \vdash M : t \rangle \preceq^{\Gamma, t} [\langle \Gamma \vdash M : t \rangle]$$

The two parts are proved simultaneously by induction on typing derivations, as in most logical relations proofs. In the case for recursive functions, there is a nested induction over the step indices.

Theorem 2.

1. For all Γ, V, t , if $\Gamma \vdash V : t$ then

$$\langle \Gamma \vdash V : t \rangle \succeq^{\Gamma, t} [\langle \Gamma \vdash V : t \rangle]$$
2. For all Γ, M, t , if $\Gamma \vdash M : t$ then

$$\langle \Gamma \vdash M : t \rangle \succeq^{\Gamma, t} [\langle \Gamma \vdash M : t \rangle]$$

This is another simultaneous induction on typing derivations. This time, the proof for recursive functions involves showing that each of the domain elements in the chain whose limit is the denotation is in the relation and then concluding that the fixpoint is in the relation by admissibility.

Corollary 1.

1. For all Γ, V, t , if $\Gamma \vdash V : t$ then

$$\langle \Gamma \vdash V : t \rangle \models^{\Gamma, t} [\langle \Gamma \vdash V : t \rangle]$$
2. For all Γ, M, t , if $\Gamma \vdash M : t$ then

$$\langle \Gamma \vdash M : t \rangle \models^{\Gamma, t} [\langle \Gamma \vdash M : t \rangle]$$

So the compiled code of a well-typed term always realizes the denotational semantics of that term. A consequence is that compiled code for whole programs has the correct operational behaviour according to the denotational semantics:

Corollary 2.

- If $\langle \langle \rangle \vdash M : \text{Int} \rangle = \perp$ then $\langle \langle \rangle \vdash M : \text{Int} \rangle$ diverges unconditionally.
- If $\langle \langle \rangle \vdash M : \text{Int} \rangle = [n]$ then $\langle \langle \rangle \vdash M : \text{Int} \rangle$ converges to n .

which follows by the adequacy lemmas above. And of course, by composing with the result that the denotational semantics is adequate with respect to the operational semantics of the source language, one obtains another corollary, that the operational semantics of complete source programs agrees with that of their compiled versions. It is this last corollary that is normally thought of as compiler correctness, but it is Corollary 1 that is really interesting, as it is that which allows us to reason about the combination of compiled code with code from elsewhere.

6.2 Low-level Equational Reasoning

In this section we give some simple examples of typed equivalences one can prove on low-level code. We first define some macros for composing SECD programs. If $c, cf, cx \in \text{Code}$ then define

$$\begin{aligned} \text{LAMBDA}(c) &= [\text{PushC}(c \text{ ++ } [\text{Ret}])] \\ \text{APP}(cf, cx) &= cf \text{ ++ } cx \text{ ++ } [\text{App}] \end{aligned}$$

6.2.1 Example: Commutativity of addition

Define the following source term

$$\text{plussrc} = (\text{Fix } f \ x = [\text{Fix } g \ y = x + y])$$

and its compiled code

$$\text{pluscode}(\Gamma) = \langle \Gamma \vdash \text{plussrc} : \text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \rangle.$$

Now we can show the following:

Lemma 8. For any Γ , for any $c_1, c_2 \in \text{Code}$, and $dc_1, dc_2 \in \langle \Gamma \rangle \Rightarrow \langle \text{Int} \rangle_{\perp}$, if

$$c_1 \models^{\Gamma, \text{Int}} dc_1 \quad \text{and} \quad c_2 \models^{\Gamma, \text{Int}} dc_2$$

then

$$\begin{aligned} \Gamma \vdash \text{APP}(\text{APP}(\text{pluscode } \Gamma, c_1), c_2) \\ \sim \text{APP}(\text{APP}(\text{pluscode } \Gamma, c_2), c_1) : \text{Int} \end{aligned}$$

In other words, for any code fragments c_1, c_2 that are in the interpretation of the source language type Int , manually composing those fragments with the code produced by the compiler for the curried addition function in either order yields equivalent behaviour of type Int .

6.2.2 Example: First projection

Define the source term

$$\text{projfstsrc} = (\text{Fix } f \ x = [\text{Fix } g \ y = x])$$

and the compiled code

$$\text{projfstcode}(\Gamma, t, t') = \langle \Gamma \vdash \text{projfstsrc} : t \rightarrow t' \rightarrow t \rangle$$

then

Lemma 9. For any Γ, t, t' , for any $c_1, c_2 \in \text{Code}$ and $dc_1 \in \langle \Gamma \rangle \Rightarrow \langle t \rangle_{\perp}$ and $dc_2 \in \langle \Gamma \rangle \Rightarrow \langle t' \rangle_{\perp}$, if

$$c_1 \models^{\Gamma, t} dc_1 \quad \text{and} \quad c_2 \models^{\Gamma, t'} dc_2$$

and furthermore

$$\forall de \in \langle \Gamma \rangle, \exists dv \in \langle t' \rangle (dc_2 \ de) = [dv]$$

which says that the code c_2 realizes some total denotational computation of type t' in context Γ , then

$$\Gamma \vdash \text{APP}(\text{APP}(\text{projfstcode}(\Gamma, t, t'), c_1), c_2) \sim c_1 : t.$$

This says that the compiled version of projfstsrc behaves like the first projection, provided that the second argument does not diverge.

6.2.3 Example: Optimizing iteration

Our last example is slightly more involved, and makes interesting use of the non-functional equality test in the target language. We start by compiling the identity function on integers

$$\begin{aligned} \text{idsrc} &= \text{Fix } \text{id} \ x = x \\ \text{idcode}(\Gamma) &= \langle \Gamma \vdash \text{idsrc} : \text{Int} \rightarrow \text{Int} \rangle \end{aligned}$$

and then define a higher-order function appnsrc that takes a function f from integers to integers, an iteration count n and an integer v , and returns f applied n times to v . We present the definition in ML-like syntax rather than our ANF language to aid readability:

$$\begin{aligned} \text{appnsrc} &= \text{fun } f \Rightarrow \\ &\quad \text{letrec apf } n = \\ &\quad \text{fun } v \Rightarrow \\ &\quad \quad \text{if } n > 0 \\ &\quad \quad \text{then } f \ (\text{apf } (n-1) \ v) \\ &\quad \quad \text{else } v \end{aligned}$$

and we let

$$\text{appnocode}(\Gamma) = (\Gamma \vdash \text{appnsrc} : (\text{Int} \rightarrow \text{Int}) \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Int})$$

Now we define a handcrafted optimized version, $\text{appnoptcode}(\Gamma)$ in the SECD language, which would, if one could write it, correspond to ML-like source code looking something like this:

```
fun f => fun n => fun v =>
  if f =α idcode(Γ)
  then v
  else appnsrc f n v
```

The optimized code checks to see if it has been passed the literal closure corresponding to the identity function, and if so simply returns v without doing any iteration. We are then able to show that for any Γ ,

$$\Gamma \vdash \text{appnoptcode}(\Gamma) \sim \text{appnocode}(\Gamma) : (\text{Int} \rightarrow \text{Int}) \rightarrow \text{Int} \rightarrow \text{Int}$$

showing that the optimized version, which could not be written in the source language, is equivalent to the unoptimized original.

7. Discussion

We have given a realizability relation between the domains used to model a simply-typed functional language with recursion and the low level code of an SECD machine with non-functional features. This relation was used to establish a semantic compiler correctness result and to justify typed equational reasoning on handcrafted low-level code. The relations make novel use of biorthogonality and step-indexing, and the work sheds interesting new light on the interaction of these two useful constructions.

As we said in the introduction, there are many other compiler correctness proofs in the literature, but they tend not to be so compositional or semantic in character as the present one. The classic work on the VLISP verified Scheme compiler by Guttman et al. (1995) is very close in character, being based on relating a denotational semantics to, ultimately, real machine code. The untyped PreScheme language treated in that work is significantly more realistic than the toy language of the present work, though the proofs were not mechanized. The denotational semantics used there was in CPS and the main emphasis was on the behaviour of complete programs. Chlipala (2007) has also used Coq to formalize a correctness relation between a high-level functional language and low-level code, though in that case the source language is total and so can be given an elementary semantics in Sets.

For ML-like languages (pure or impure), contextual equivalence at higher types is highly complex, depending subtly on exactly what primitives one allows. This is why, as we said in the introduction, we feel that fully abstract compilation might not be quite the right thing to aim for. For example, Longley (1999) shows that there are extensionally pure (and even useful) functionals that are only definable in the presence of impure features, such as references. Adding such functionals is not conservative – they refine contextual equivalence at order four and above in pure CBV languages – yet it seems clear that they will be implementable in many low-level machines. Complicating one’s specifications to rule out these exotic programs in pursuit of full abstraction is not obviously worthwhile: it seems implausible that any compiler transformations or information flow policies will be sensitive to the difference. As another example, the presence of strong reflective facilities at the low-level, such as being able to read machine code instructions, might well make parallel-or definable; this would obviously break full

abstraction with respect to the source language, but we might well wish to allow it.⁴

The case against full abstraction becomes stronger when one considers that one of our aims is to facilitate semantically type safe linking of code produced from *different* programming languages. There is a fair degree of ‘wiggle room’ in deciding just how strong the semantic assumptions and guarantees should be across these interfaces. They should be strong enough to support sound and useful reasoning from the point of view of one language, but not insist on what one might almost think of as ‘accidental’ properties of one language that might be hard to interpret or ensure from the point of view of others.

The Coq formalization of these results was pleasantly straightforward. Formalizing the SECD machine, compiler, logical relations and examples, including the compiler correctness theorem, took a little over 4000 lines, not including the library for domain theory and the semantics of the source language. The extra burden of mechanized proof seems fairly reasonable in this case, and the results, both about the general theory and the examples, are sufficiently delicate that our confidence in purely paper proofs would be less than complete.

There are many obvious avenues for future work, including the treatment of richer source languages and type systems, and of lower-level target languages. We intend particularly to look at source languages with references and polymorphism and at a target machine like the idealized assembly language of our previous work. We would also like to give low-level specifications that are more independent of the source language – the current work doesn’t mention source language terms, but does still talk about particular cpos. We would like to express essentially the same constraints in a more machine-oriented relational Hoare logic which might be more language neutral and better suited for independent verification. It would also be interesting to look at type systems that are more explicitly aimed at ensuring secure information flow.

Acknowledgments

Thanks to Andrew Kennedy, Neel Krishnaswami, Nicolas Tabareau and Carsten Varming for many useful discussions. Extra thanks to Andrew and Carsten for their work on the Coq formalization of the source language and its denotational semantics.

References

- M. Abadi. $\top\top$ -closed relations and admissibility. *Mathematical Structures in Computer Science*, 10(3), 2000.
- M. Abadi. Protection in programming-language translations. In *25th International Colloquium on Automata, Languages and Programming (ICALP)*, volume 1443 of *Lecture Notes in Computer Science*, 1998.
- A. Ahmed. Step-indexed syntactic logical relations for recursive and quantified types. In *15th European Symposium on Programming (ESOP)*, volume 3924 of *Lecture Notes in Computer Science*, 2006.
- A. Ahmed and M. Blume. Typed closure conversion preserves observational equivalence. In *13th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2008.
- A. Ahmed, D. Dreyer, and A. Rossberg. State-dependent representation independence. In *36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2009.
- A. Appel and D. McAllester. An indexed model of recursive types for foundational proof-carrying code. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 23(5), 2001.
- A.W. Appel, P.-A. Mellies, C.D. Richards, and J. Vouillon. A Very Modal Model of a Modern, Major, General Type System. *34th ACM SIGPLAN-*

⁴Discussing the sense in which these two candidate full abstraction-breaking operations are incompatible would take us too far afield, however.

SIGACT Symposium on Principles of Programming Languages (POPL), 2007.

- N. Benton. Abstracting allocation: The new new thing. In *20th International Workshop on Computer Science Logic (CSL)*, volume 4207 of *LNCS*, 2006.
- N. Benton and N. Tabareau. Compiling functional types to relational specifications for low level imperative code. In *4th ACM SIGPLAN Workshop on Types in Language Design and Implementation (TLDI)*, 2009.
- N. Benton and U. Zarfaty. Formalizing and verifying semantic type soundness of a simple compiler. In *9th ACM SIGPLAN International Symposium on Principles and Practice of Declarative Programming (PPDP)*, 2007.
- N. Benton, A. Kennedy, and C. Varming. Some domain theory and denotational semantics in Coq. In *22nd International Conference on Theorem Proving in Higher Order Logics (TPHOLs)*, volume 5674 of *Lecture Notes in Computer Science*, 2009.
- A. Chlipala. A certified type-preserving compiler from lambda calculus to assembly language. In *ACM SIGPLAN 2007 Conference on Programming Language Design and Implementation (PLDI)*, 2007.
- M. Dave. Compiler verification: a bibliography. *ACM SIGSOFT Software Engineering Notes*, 28(6), 2003.
- J. Guttman, J. Ramsdell, and M. Wand. VLISP: A verified implementation of scheme. *Lisp and Symbolic Computation*, 8(1/2), 1995.
- T. Hardin, L. Maranget, and B. Pagano. Functional runtime systems within the lambda-sigma calculus. *Journal of Functional Programming*, 8, 1998.
- A. Kennedy. Securing the .NET programming model. *Theoretical Computer Science*, 364(3), 2006.
- J. L. Krivine. Classical logic, storage operators and second-order lambda calculus. *Annals of Pure and Applied Logic*, 1994.
- P. Landin. The mechanical evaluation of expressions. *The Computer Journal*, 6(4), 1964.
- X. Leroy. Formal certification of a compiler back-end, or: programming a compiler with a proof assistant. In *33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2006.
- X. Leroy and H. Grall. Coinductive big-step operational semantics. *Information and Computation*, 207(2), 2009.
- J. Longley. When is a functional program not a functional program? In *4th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 1999.
- J. McCarthy and J. Painter. Correctness of a Compiler for Arithmetic Expressions. *Proceedings Symposium in Applied Mathematics*, 19:33–41, 1967.
- A. M. Pitts and I. D. B. Stark. Operational reasoning for functions with local state. In *Higher Order Operational Techniques in Semantics*. CUP, 1998.
- G. D. Plotkin. LCF considered as a programming language. *Theoretical Computer Science*, 5, 1977.
- J. Vouillon and P.-A. Mellies. Semantic types: A fresh look at the ideal model for types. In *31st ACM Symposium on Principles of Programming Languages (POPL)*, 2004.
- G. Winskel. *The Formal Semantics of Programming Languages*. MIT Press, 1993.

A. The Problem With Realizing Recursion

This appendix gives a concrete example of how defining semantics for functional types as sets of untyped programs can run into problems in the case that the source language includes recursion and the untyped target includes non-functional operations. By non-functional operations, we particularly mean operations that examine the actual syntax or code of functional terms. The example we take here is a syntactic equality test: an operation that takes two ar-

guments and returns true if the two (possibly functional) terms are syntactically equal and otherwise returns false.

Definition of VULE. As an example of a target language, we consider the call-by-value untyped lambda calculus with a syntactic equality test (VULE). We give the formal definition and operational semantics of VULE below. We fix a countable set $\mathbf{V}$ of variables. The set of values $\mathbf{Val}$ and the set of terms $\mathbf{Term}$ with variables in $\mathbf{V}$ are mutually inductively defined by the following rule:

$$\begin{array}{lcl} \mathbf{Val} & := & x \\ & | & \lambda x. t \\ \mathbf{Term} & := & v \\ & | & t s \\ & | & u \equiv_{\alpha} v \\ & | & \text{ERROR} \end{array}$$

where $x \in \mathbf{V}$, $u, v \in \mathbf{Val}$, $t, s \in \mathbf{Term}$. As usual, we assume that the application is left-associative. $\mathbf{FVar}(t)$ for any term t denotes the set of free variables in t , and $t[x \mapsto s]$ the capture-avoiding substitution of the term s for the variable x in the term t .

We also define some syntactic sugar for boolean operation and recursion (implemented via a CBV fixed point combinator).

$$\begin{aligned} \text{TRUE} &\triangleq \lambda x. \lambda y. x y \\ \text{FALSE} &\triangleq \lambda x. \lambda y. y x \\ \text{if } t \text{ then } s_1 \text{ else } s_2 &\triangleq t (\lambda x. s_1) (\lambda x. s_2) \\ &\quad (x \notin \mathbf{FVar}(s_1) \cup \mathbf{FVar}(s_2)) \\ \text{rec } t &\triangleq \lambda x. (\lambda y. t (\lambda z. y y z)) \\ &\quad (\lambda y. t (\lambda z. y y z)) x \\ &\quad (x, y \notin \mathbf{FVar}(t), z \neq y) \end{aligned}$$

The small-step call by value operational semantics of VULE is given as follows:

$$\begin{aligned} t &\rightsquigarrow t' \implies t s \rightsquigarrow t' s \\ s &\rightsquigarrow s' \implies v s \rightsquigarrow v s' \\ &\quad (\lambda x. t) v \rightsquigarrow t[x \mapsto v] \\ u \approx_{\alpha} v &\implies u \equiv_{\alpha} v \rightsquigarrow \text{TRUE} \\ u \not\approx_{\alpha} v &\implies u \equiv_{\alpha} v \rightsquigarrow \text{FALSE} \\ \text{ERROR } v &\rightsquigarrow \text{ERROR} \end{aligned}$$

where $x \in \mathbf{V}$, $u, v \in \mathbf{Val}$, $t, t', s, s' \in \mathbf{Term}$ and where $u \approx_{\alpha} v$ means that u is alpha-equivalent to v . For convenience, we define the multi-step relation $\rightsquigarrow^+$ by setting $t \rightsquigarrow^+ t'$ iff the term t reaches t' in one or more steps.

Although we have used lambda-encodings of booleans, conditionals and recursion, so as to be both concrete and minimal, the argument we shall make does not depend on these details. In fact we will only rely on some elementary properties of our encodings. The following hold for any $v \in \mathbf{Val}$ and $t, s_1, s_2 \in \mathbf{Term}$:

(Prop 1) TRUE, FALSE and $\text{rec } t$ are values;

(Prop 2) if TRUE then s_1 else $s_2 \rightsquigarrow^+ s_1$;

(Prop 3) if FALSE then s_1 else $s_2 \rightsquigarrow^+ s_2$;

(Prop 4) $(\text{rec } t) v \rightsquigarrow^+ t (\text{rec } t) v$.

The problem. We now define the problem. Let $\mathbf{Type}$ be the set of simple types defined by the rule

$$T \in \mathbf{Type} := \text{Bool} \mid T \rightarrow T.$$

As usual, the type constructor $\rightarrow$ is right-associative. Now we consider what happens if we try to give a semantics to these types as

sets of closed VULE terms that admit recursively-defined functions. Whatever the details, we would expect the following, very minimal, conditions to be satisfied by the interpretation of types $\llbracket - \rrbracket : \text{Type} \rightarrow \mathcal{P}(\text{Term})$:

(Asm 1) $\text{TRUE}, \text{FALSE} \in \llbracket \text{Bool} \rrbracket$.

(Asm 2) Given a value $u \in \llbracket A_1 \rightarrow \dots \rightarrow A_n \rightarrow B \rrbracket$ and values $v_1 \in \llbracket A_1 \rrbracket \dots v_n \in \llbracket A_n \rrbracket$, the application of u to the v_i s does not go wrong: $u \ v_1 \dots v_n \not\rightarrow \text{ERROR}$.

(Asm 3) For a value u , if $u \ v \xrightarrow{\dagger} v$ for all values $v \in \llbracket A \rrbracket$, then $u \in \llbracket A \rightarrow A \rrbracket$.

We now show that, rather distressingly, for any semantics $\llbracket - \rrbracket$ satisfying the above conditions, the fixed point combinator Y defined by

$$Y \triangleq \lambda f. \text{rec } f$$

is *not* in the set

$$\llbracket ((\text{Bool} \rightarrow \text{Bool}) \rightarrow \text{Bool} \rightarrow \text{Bool}) \rightarrow \text{Bool} \rightarrow \text{Bool} \rrbracket.$$

Let the value F be defined by

$$F \triangleq \lambda g. \lambda f. \text{if } f \equiv_{\alpha} (\text{rec } g) \text{ then ERROR else } f$$

and observe the following facts about the behaviour of F :

(Obs 1): For any value $v \in \text{Val}$,

$$\begin{aligned} (\text{rec } F) \ v &\xrightarrow{\dagger} F (\text{rec } F) \ v \\ &\xrightarrow{\dagger} \begin{cases} \text{ERROR} & \text{if } v \approx_{\alpha} \text{rec } \text{rec } F \\ v & \text{otherwise} \end{cases} \end{aligned}$$

(Obs 2): $(\text{rec } \text{rec } F) \text{ TRUE} \xrightarrow{\dagger} (\text{rec } F) (\text{rec } \text{rec } F) \text{ TRUE} \xrightarrow{\dagger} \text{ERROR} \text{ TRUE} \rightsquigarrow \text{ERROR}$

(Obs 3): $Y (\text{rec } F) \text{ TRUE} \rightsquigarrow (\text{rec } \text{rec } F) \text{ TRUE} \xrightarrow{\dagger} \text{ERROR}$

From these observations, we can conclude that

(Con 1): $\text{rec } \text{rec } F \notin \llbracket \text{Bool} \rightarrow \text{Bool} \rrbracket$ because $\text{TRUE} \in \llbracket \text{Bool} \rrbracket$

(Asm 1), $(\text{rec } \text{rec } F) \text{ TRUE} \xrightarrow{\dagger} \text{ERROR}$ (Obs 2), and well-typed applications don't go wrong (Asm 2).

(Con 2): $\text{rec } F \in \llbracket (\text{Bool} \rightarrow \text{Bool}) \rightarrow \text{Bool} \rightarrow \text{Bool} \rrbracket$ because $\text{rec } F$ behaves as the identity on all values except $\text{rec } \text{rec } F$

(Obs 1), and since $\text{rec } \text{rec } F$ is *not* in $\llbracket \text{Bool} \rightarrow \text{Bool} \rrbracket$ (Con 1), $\text{rec } F$ must behave as the identity on all values that are in $\llbracket \text{Bool} \rightarrow \text{Bool} \rrbracket$, we get the conclusion by (Asm 3).

(Con 3): Now we see that

$$Y \notin \llbracket ((\text{Bool} \rightarrow \text{Bool}) \rightarrow \text{Bool} \rightarrow \text{Bool}) \rightarrow \text{Bool} \rightarrow \text{Bool} \rrbracket$$

because

$$\begin{aligned} Y (\text{rec } F) \text{ TRUE} &\xrightarrow{\dagger} \text{ERROR} && \text{(Obs 3)} \\ \text{rec } F \in \llbracket (\text{Bool} \rightarrow \text{Bool}) \rightarrow \text{Bool} \rightarrow \text{Bool} \rrbracket &&& \text{(Con 2)} \\ \text{TRUE} \in \llbracket \text{Bool} \rrbracket &&& \text{(Asm 1)} \end{aligned}$$

and well-typed applications don't go wrong (Asm 2).

This is not at all what we wanted! One would expect (Con 2) to be false, since F is clearly highly suspicious, and then the blameless Y could have the expected type. Our analysis of the problem is that (Asm 3) is the only one of our assumptions that could be modified in order to get the expected result. In short, just testing with 'good' arguments is actually insufficient grounds for concluding that a function is good: we need some extra tests on 'partially good' values, which is just what step-indexing will supply.

B. On the non-closure of $\geq^t$

The reason $\{d \mid v \geq^t d\}$ is not always closed under taking limits of chains is essentially that $(\cdot)^{\perp\perp}$ does not preserve meets. In particular

$$\bigcap_i (P_i^{\top\top}) = \left(\bigcup_i (P_i^{\top}) \right)^{\top} \supset \left(\bigcap_i P_i \right)^{\top\top}$$

with the last inclusion following from contravariance and the strict inclusion

$$\bigcup_i (P_i^{\top}) \subset \left(\bigcap_i P_i \right)^{\top}.$$

Now let $\langle df_i \rangle$ be a chain of elements of $\llbracket t_1 \rightarrow t_2 \rrbracket$ with $\sqcup_i df_i = df$, such that for all i and for all $dv \in \llbracket t_1 \rrbracket$, $df_i \ dv \sqsupset \perp$. Then we claim that in general, there is a strict inclusion

$$\{f \mid \forall i, f \geq^{t_1 \rightarrow t_2} df_i\} \supset \{f \mid f \geq^{t_1 \rightarrow t_2} df\}$$

Expanding the definitions, f being in the right-hand set above means exactly that for all v, dv such that $v \geq^{t_1} dv$,

$$(([\text{App}], [v, f]), \lambda *. df \ dv) \in (I \xrightarrow{\triangleright} \geq_{\perp}^{t_2})$$

which expands to

$$\forall d, d = [df \ dv] \Rightarrow ([\text{App}], [v, f]) \in \overline{\uparrow}(\downarrow(\{w \mid w \geq^{t_2} d\}))$$

or, equivalently (because of the non- $\perp$ assumption),

$$([\text{App}], [v, f]) \in \overline{\uparrow}(\downarrow(\{w \mid \forall d, d = [df \ dv] \Rightarrow w \geq^{t_2} d\}))$$

which is to say, $([\text{App}], [v, f])$ is in

$$\overline{\uparrow}(\downarrow(\{\widehat{w} \mid \forall d, d = [df \ dv] \Rightarrow w \geq^{t_2} d\})). \quad (1)$$

Using similar reasoning, one can deduce that f being in the left-hand side of our inclusion is equivalent to saying that for all v, dv such that $v \geq^{t_1} dv$, the computation $([\text{App}], [v, f])$ is in

$$\bigcap_i \overline{\uparrow}(\downarrow(\{\widehat{w} \mid \forall d, d = [df_i \ dv] \Rightarrow w \geq^{t_2} d\})). \quad (2)$$

But there is in general a strict inclusion between the set in (1) and that in (2). By down-closure,

$$\begin{aligned} &\{\widehat{w} \mid \forall d, d = [df \ dv] \Rightarrow w \geq^{t_2} d\} \\ &\subseteq \bigcap_i \{\widehat{w} \mid \forall d, d = [df_i \ dv] \Rightarrow w \geq^{t_2} d\} \end{aligned}$$

so

$$\begin{aligned} &\overline{\uparrow}(\downarrow(\{\widehat{w} \mid \forall d, d = [df \ dv] \Rightarrow w \geq^{t_2} d\})) \\ &\subseteq \overline{\uparrow}(\downarrow(\bigcap_i \{\widehat{w} \mid \forall d, d = [df_i \ dv] \Rightarrow w \geq^{t_2} d\})) \\ &\subset \bigcap_i \overline{\uparrow}(\downarrow(\{\widehat{w} \mid \forall d, d = [df_i \ dv] \Rightarrow w \geq^{t_2} d\})) \end{aligned}$$

by the general property of biorthogonals above.

To address this issue, we can either add in 'just enough' limits or close up under *all* limits. The first version of our definitions looked like

$$v \succeq^t dv \stackrel{\text{def}}{\iff} \exists \langle d_i \rangle, dv \sqsubseteq \bigsqcup_i d_i \wedge \forall i, v \geq^t d_i$$

which adds limits of arbitrary chains to (and down-closes) the right-hand side of the relation. The resulting relation may still not be closed under limits of chains, but actually works perfectly well for our purposes, as it does contain limits of all chains arising in the semantics of functions in our language. (And we even proved all the theorems in Coq using this definition. . .) Nevertheless, it seems mathematically more natural to work with Scott-closed sets, so we have now adopted the definition using $\text{Clos}(\cdot)$ shown in the main text.